

Sprog og oversættelse

Lexical analysis: From NFA to DFA; Minimizing a DFA

Part 2: DFA Minimization

Hans Hüttel

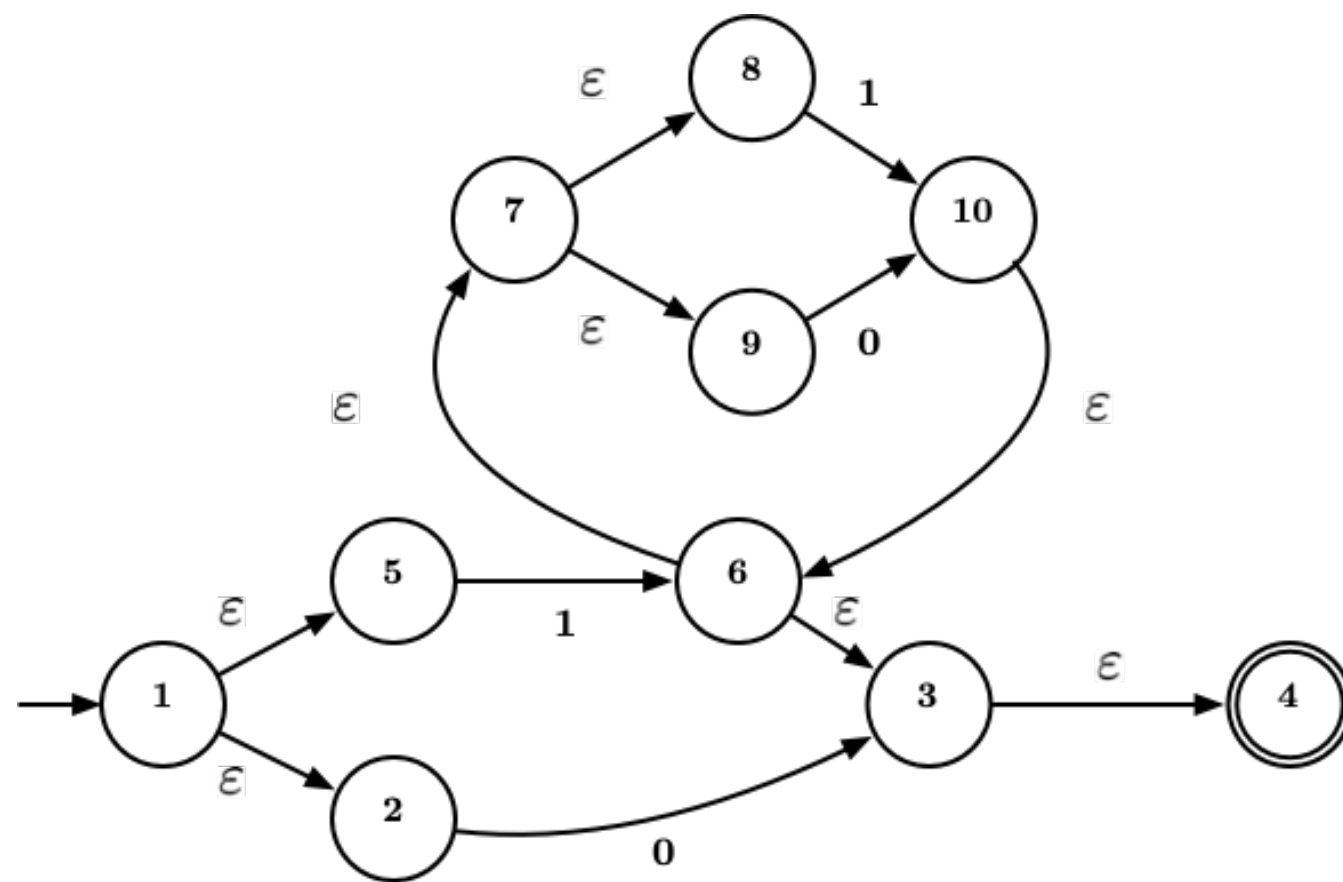
(using content by Cosmin Oancea)

Creative Commons License
Attribution Non Commercial 4.0 International
(CC-BY-NC-4.0)

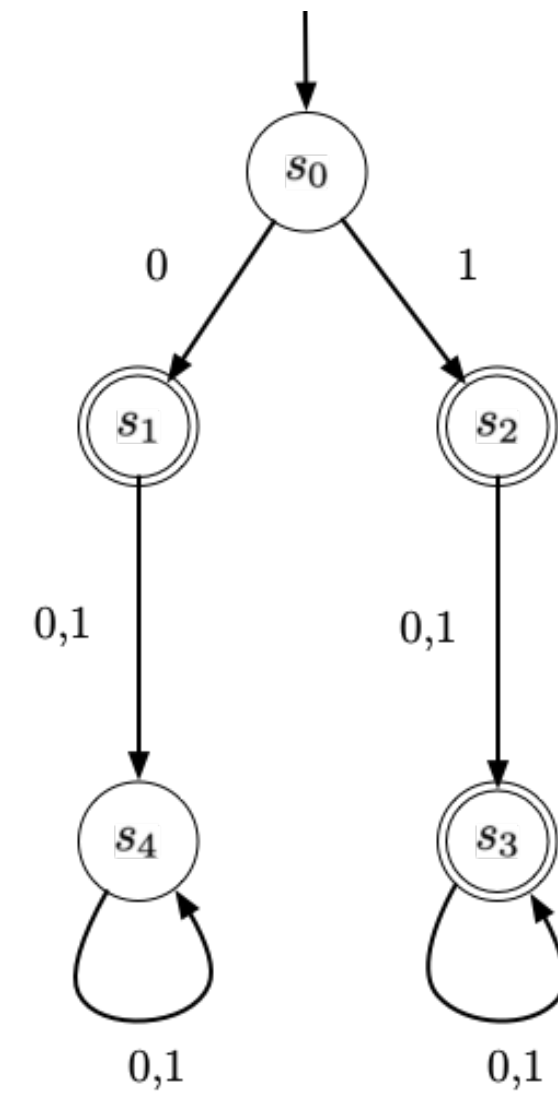
Some DFAs are needlessly large!

There are times when the DFA that we get from the subset construction can be collapsed into a DFA that is simpler.

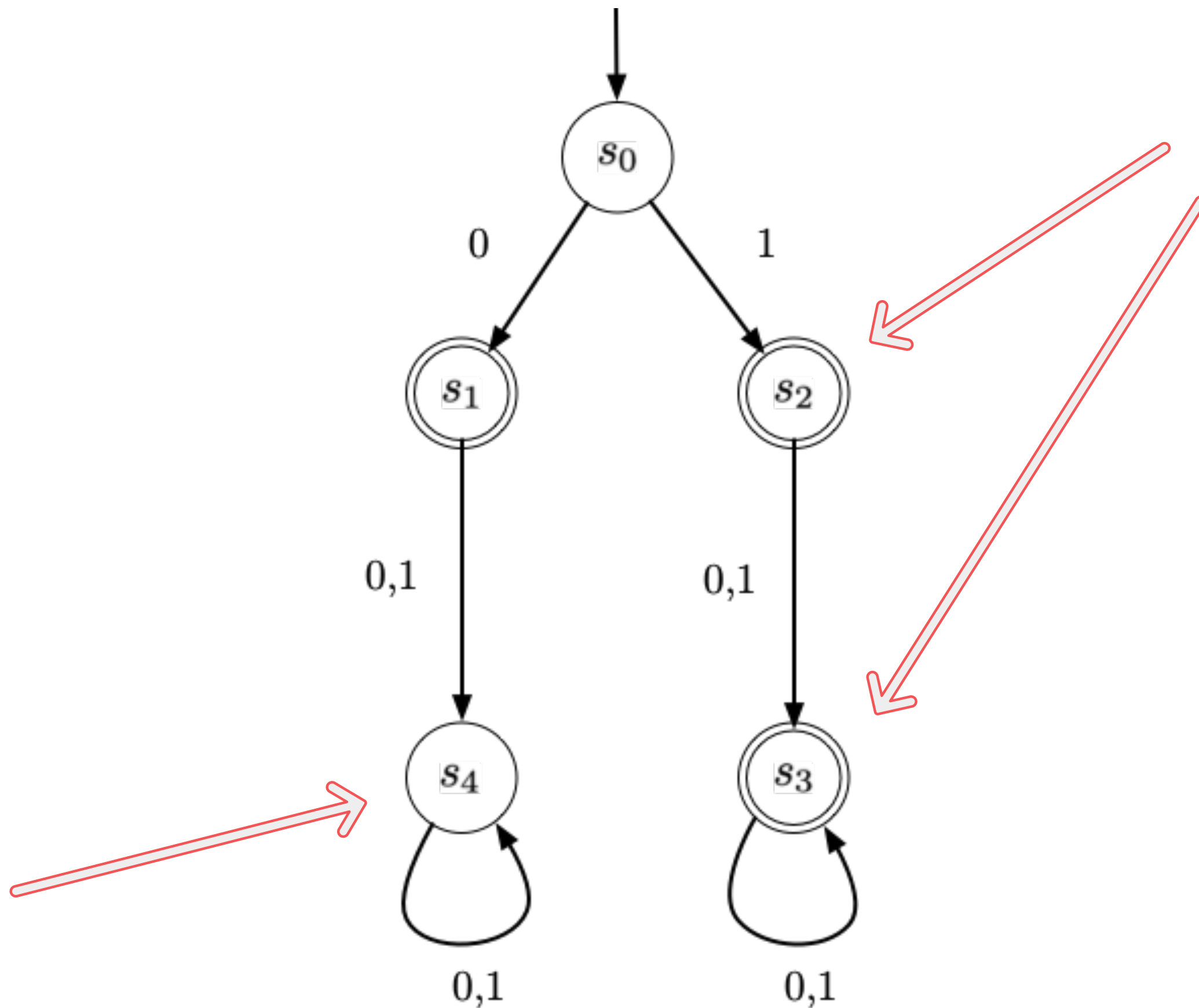
NFA



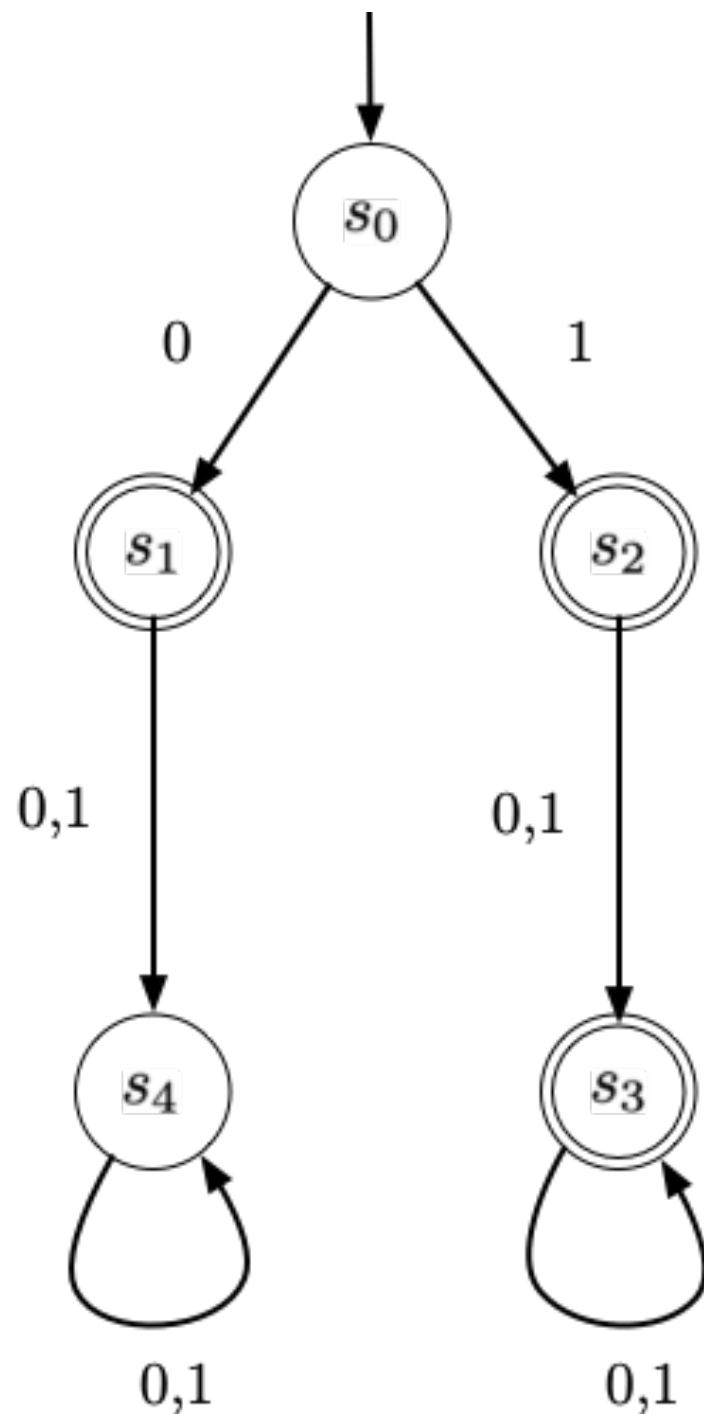
DFA



This DFA is slightly too large

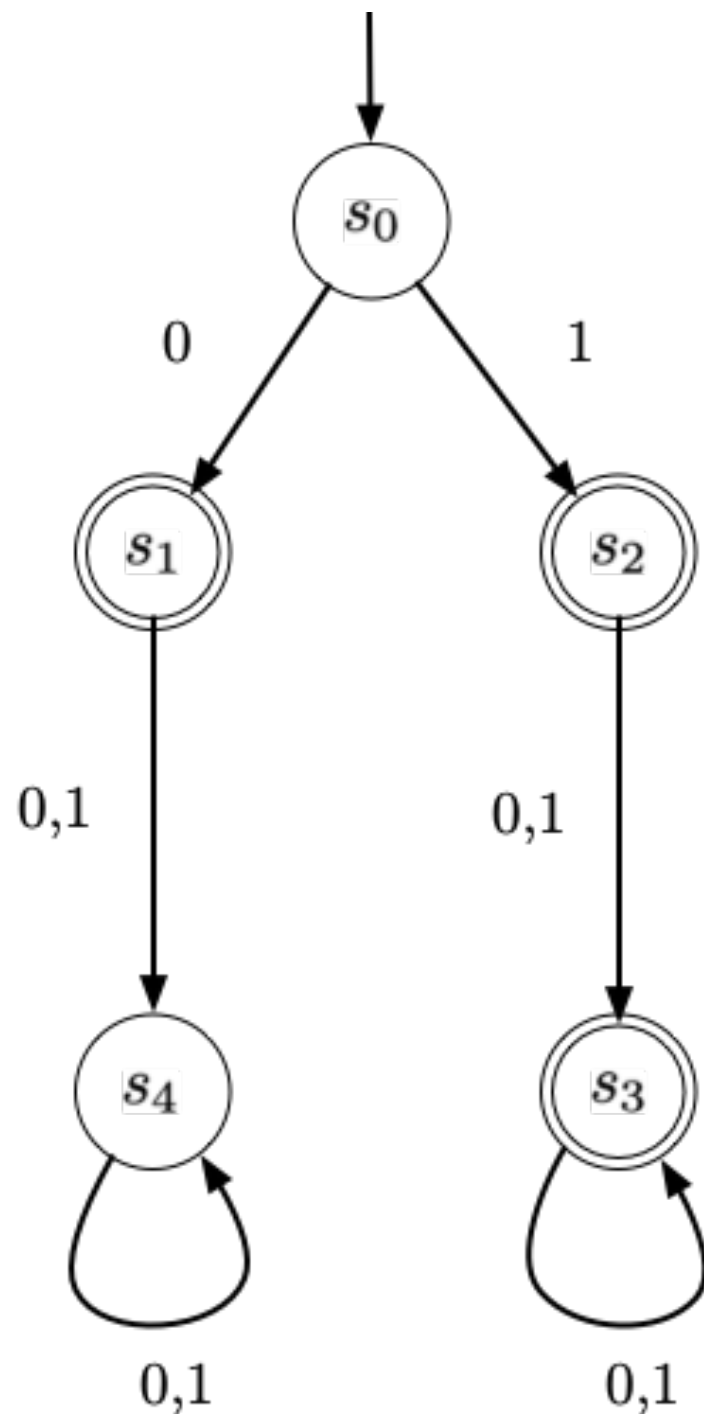


Minimizing a DFA



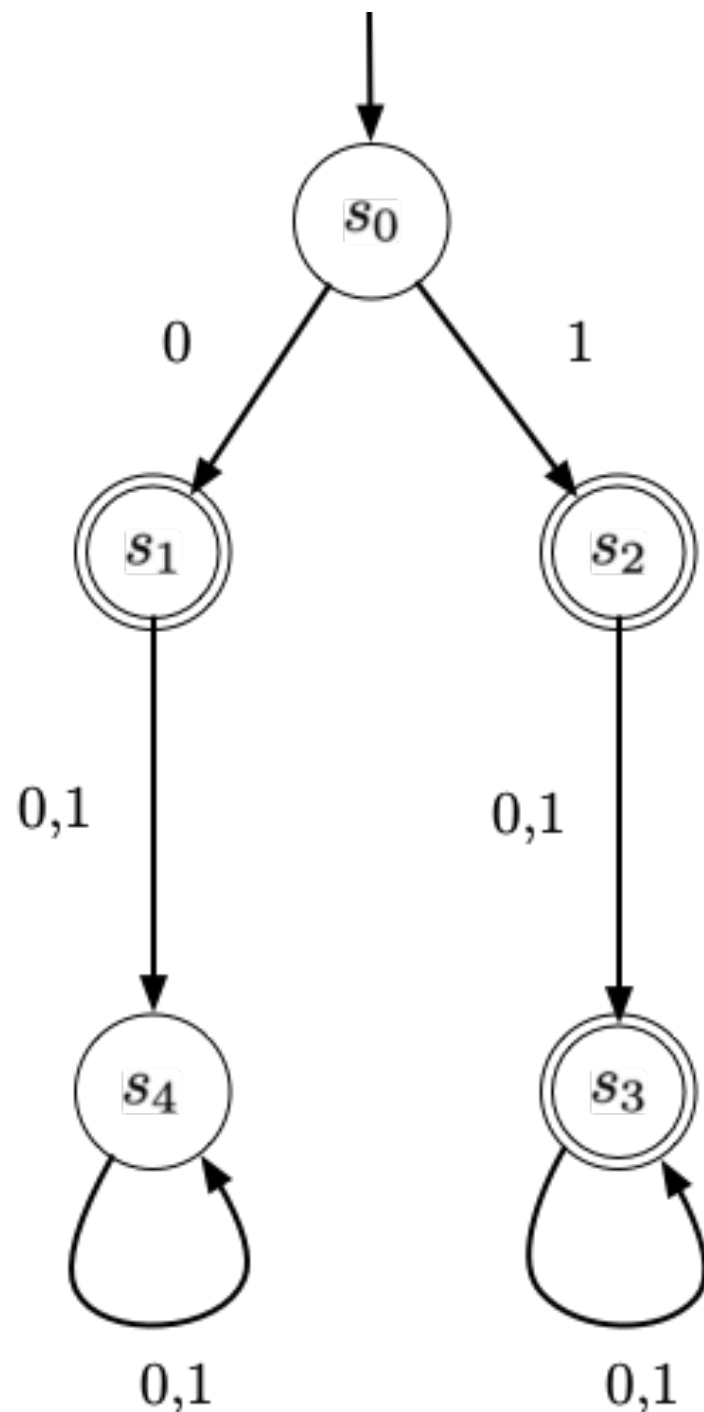
- We would like to get rid of **dead states**. These are states that cannot lead to a final state with **any** input.

Minimizing a DFA



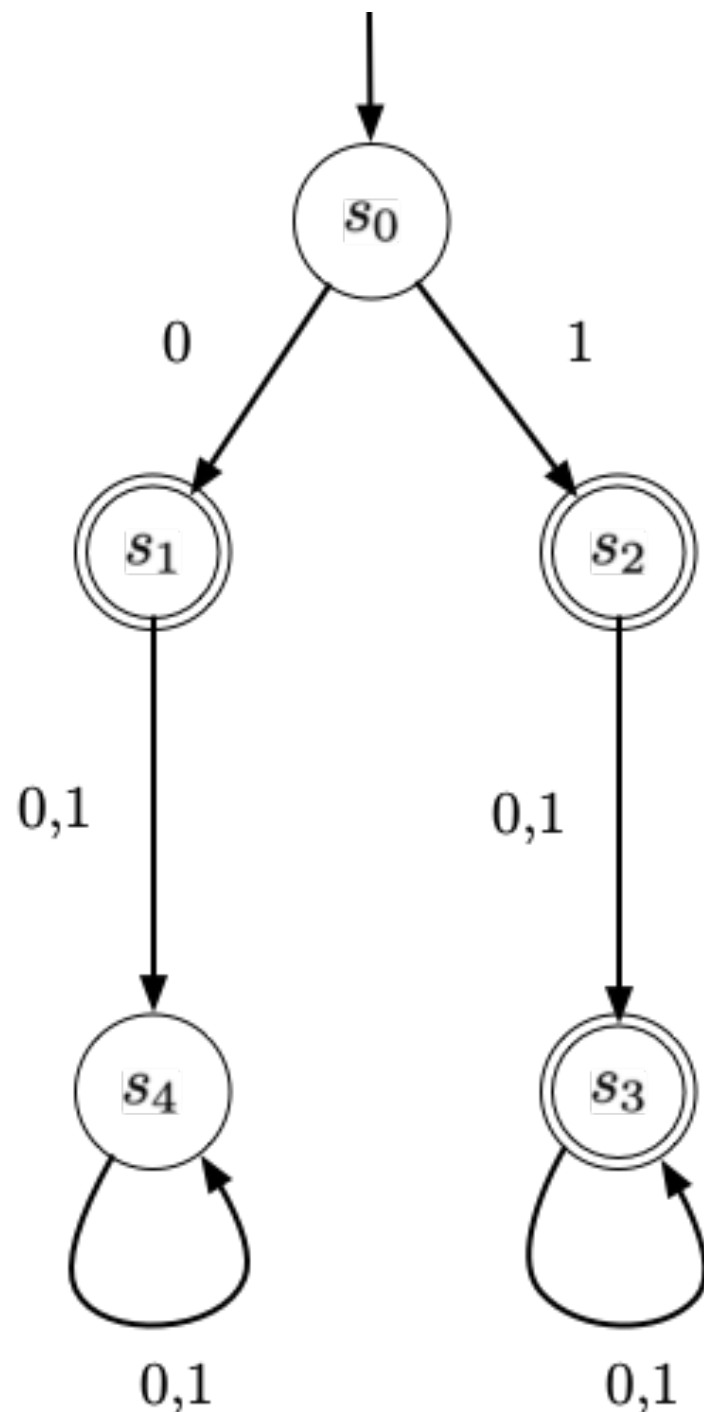
- We would like to get rid of **dead states**. These are states that cannot lead to a final state with **any** input. s_4 is dead.

Minimizing a DFA



- We would like to get rid of **dead states**. These are states that cannot lead to a final state with **any** input. s_4 is dead.
- We would like to collapse **equivalent states** into the same state. Equivalent states lead to acceptance for the same strings.

Minimizing a DFA



- We would like to get rid of **dead states**. These are states that cannot lead to a final state with **any** input. s_4 is dead.
- We would like to collapse **equivalent states** into the same state. Equivalent states lead to acceptance for the same strings. s_2 and s_3 are equivalent.

Equivalent states

Definition

Let D be a DFA with state set S and final states F . The language recognized by s is

$$L(s) = \{w \mid s \xrightarrow{w} s_f \text{ for some } s_f \in F\}$$

This definition tells us that if s_0 is the start state of D , then $L(D) = L(s_0)$.

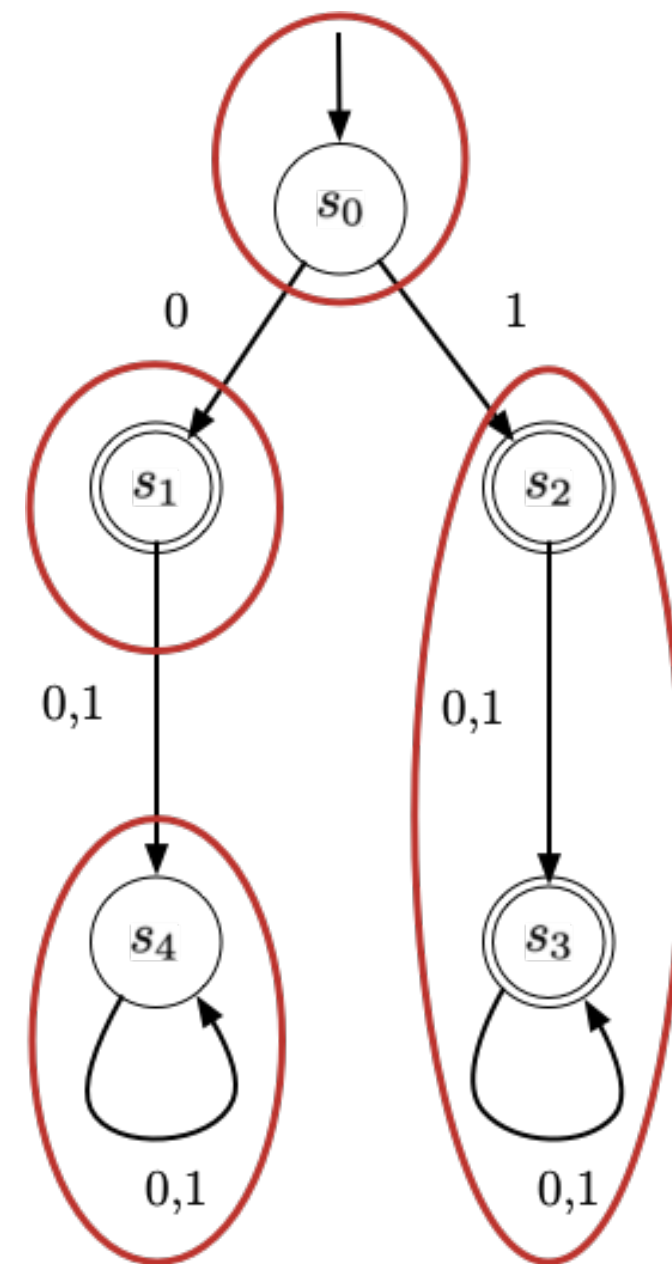
We say that two states s_1 and s_2 are equivalent if $L(s_1) = L(s_2)$ and write $s_1 \sim s_2$ if this is the case.

The idea behind the minimization algorithm

The algorithm will partition the set of states of our DFA into a family of groups of states such that

- Whenever $s_1, s_2 \in G$ then $s_1 \sim s_2$
- If $s_1 \in G_1$ and $s_2 \in G_2$ and $G_1 \neq G_2$ then $s_1 \not\sim s_2$

These groups are called *equivalence classes* of $\sim$. Our new DFA then has one state for each equivalence class and transitions from the old DFA (remember, all states in an equivalence class have the same transitions!)



The idea behind the minimization algorithm

We start with two groups of states G_1 and G_2 that must be separate – the set of accepting states and the set of non-accepting states. Clearly, if $s_1 \in F$ and $s_2 \in S \setminus F$, then $s_1 \not\sim s_2$.

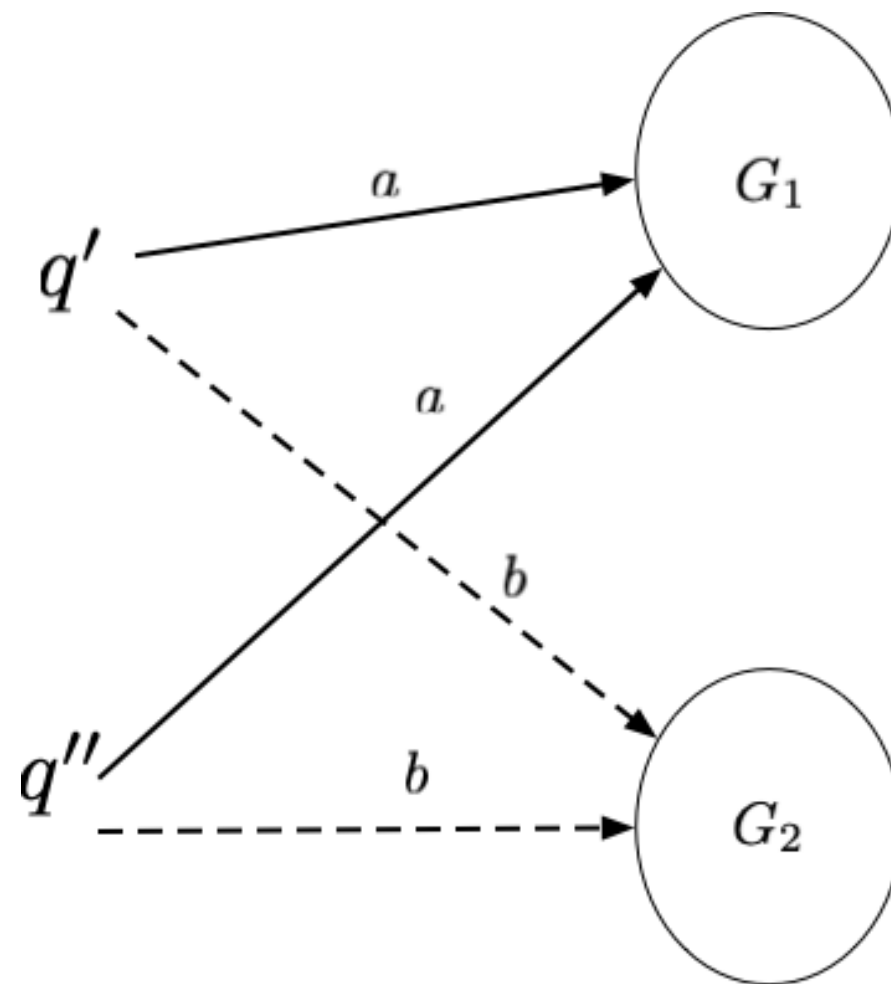
The algorithm is a loop that in every iteration will try to **split groups**.

Before and after each iteration, we know that

- states in the same group *may* be equivalent
- states from different groups cannot be equivalent

When we can no longer split any groups, we have found the equivalence classes.

Splitting a group

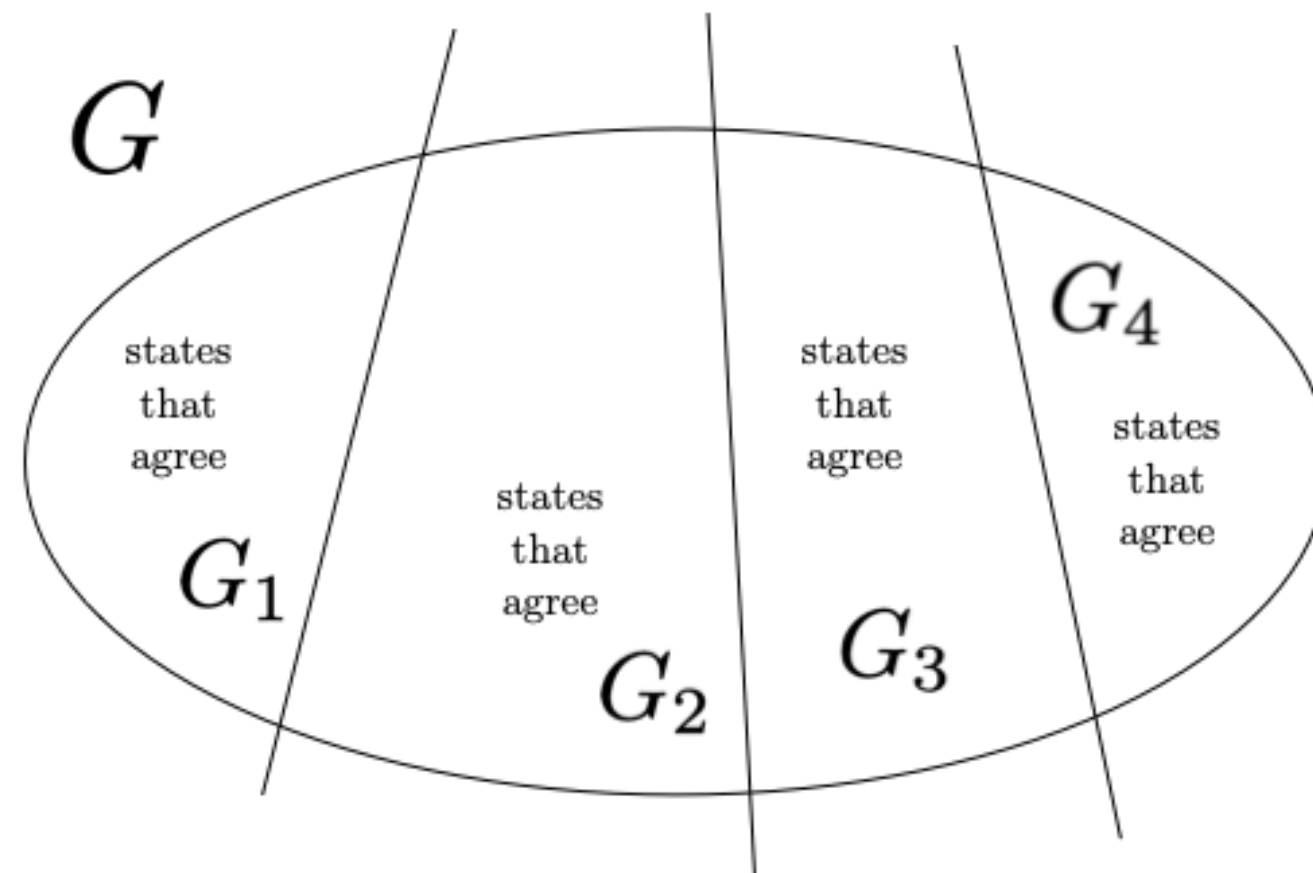


Given a partition of our set of states, two states q' and q'' **agree** if for any character a we have that their outgoing a -transitions always agree on which group they go to.

If $q' \sim q''$, they must agree! (The converse may not be the case.)

Splitting a group G

- $i := 1$
- While there are states from G that have not been placed in a new group
 - Pick a state $q \in G$ that has not yet been placed in a new group
 - Find all the states that agree with q . This collection of states is the new group G_i
 - $i := i + 1$



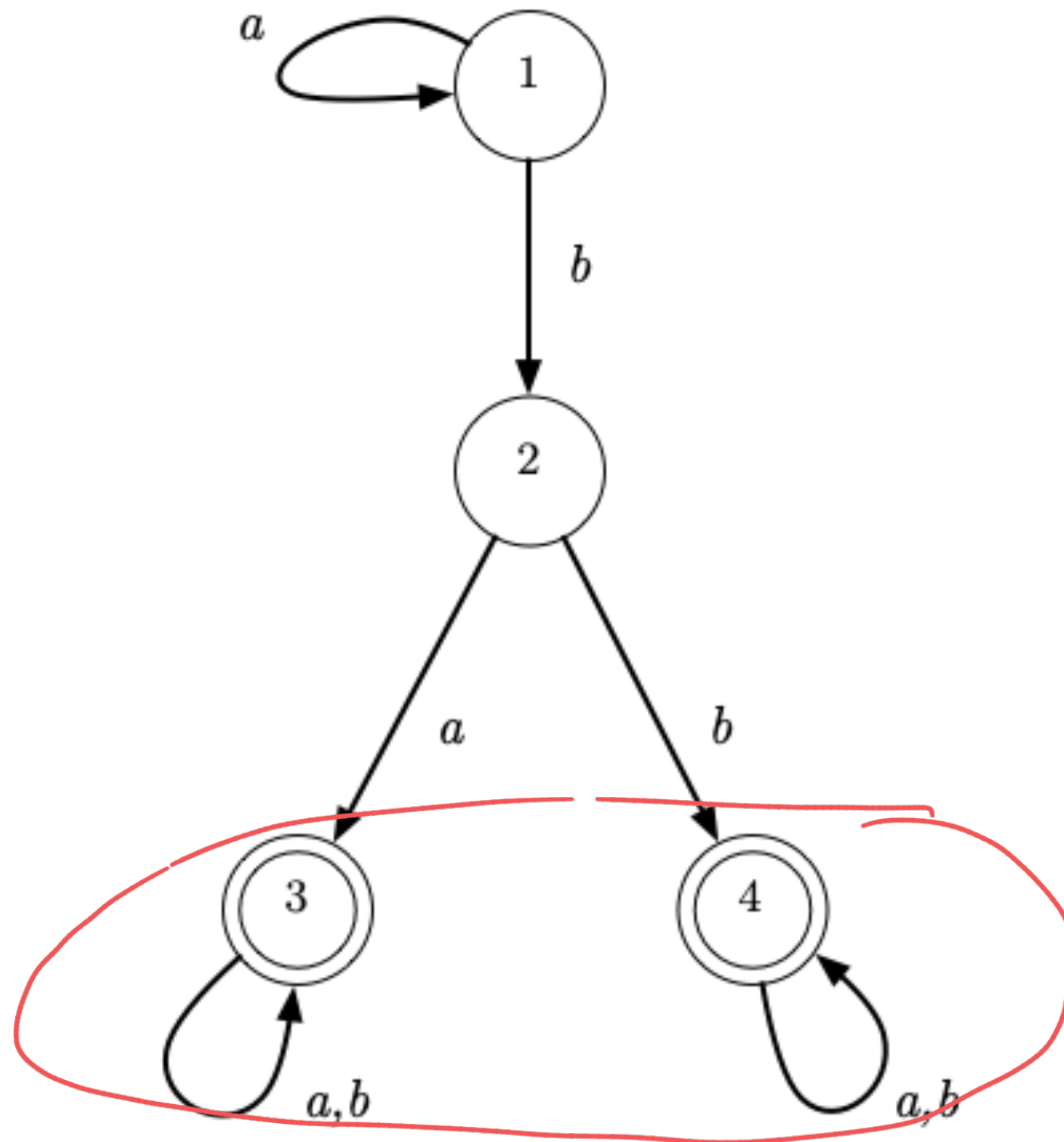
The DFA minimization algorithm

Let $D = (S, \Sigma, s_0, F, \{\xrightarrow{a} \mid a \in \Sigma\})$ be a DFA.

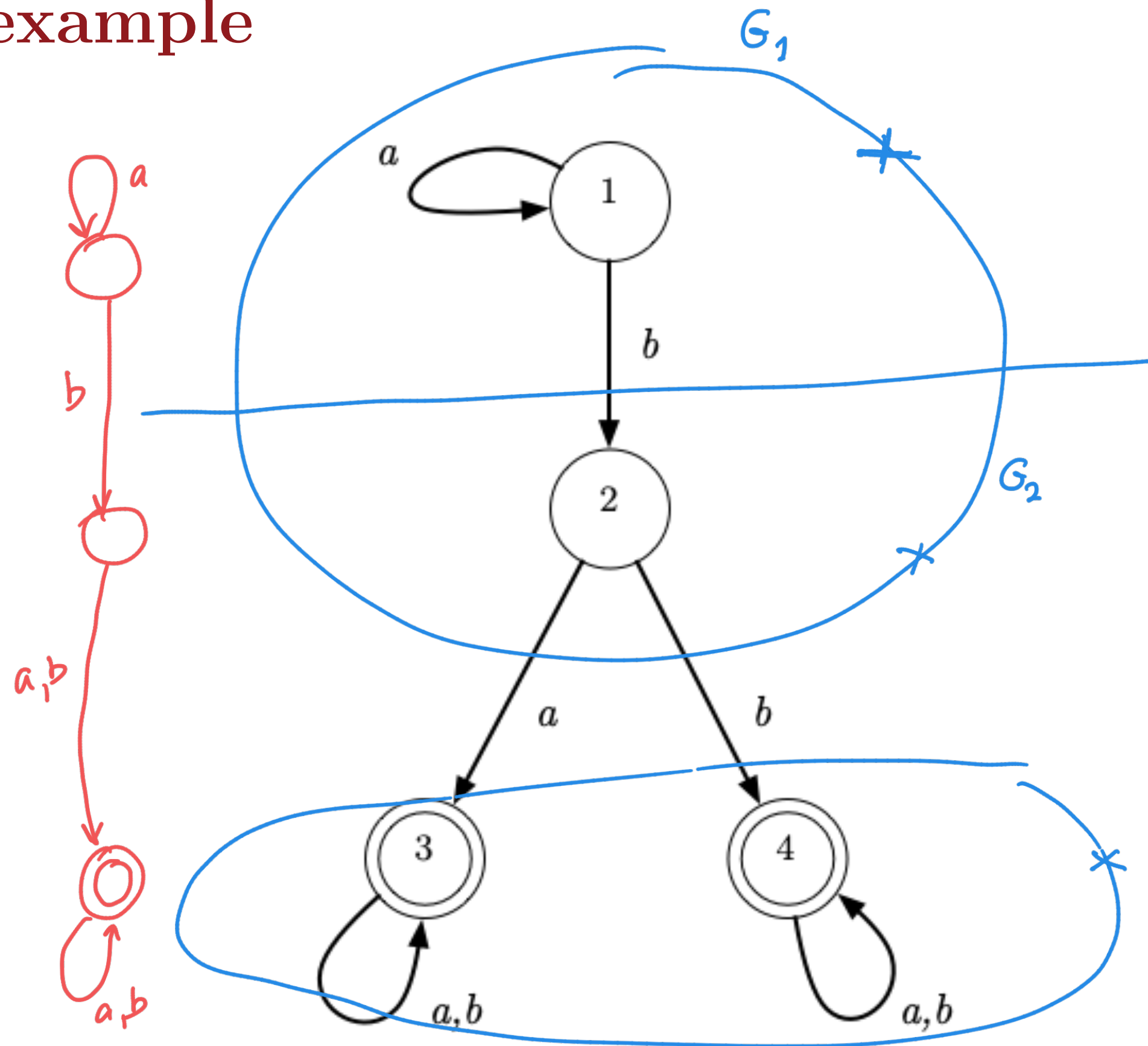
- 1 Start with two unmarked groups, F and $S \setminus F$.
- 2 While there exist unmarked groups:
 - a. pick an unmarked group G
 - b. if G cannot be split, then mark the group
 - c. else split the group G into new groups and unmark all groups!
- 3 Repeat from 2 until all the groups are marked.

The resulting groups contain equivalent states!

An example



An example



Some assumptions we made

The minimization algorithm assumes that

- there are no dead states in the DFA, or
- that the transition relation is total.

In the presence of both, the minimization might fail!

What if the transition relation is not total?

Some assumptions we made

The minimization algorithm assumes that

- there are no dead states in the DFA, or
- that the transition relation is total.

In the presence of both, the minimization might fail!

What if the transition relation is not total?

Make it total by adding a default dead state.

All dead states are equivalent now and can be removed!

The resulting groups contain equivalent states!